Library Management System (LibraryDB)

DBMS Mini Project | **Name:** Aditya Kumar | **Roll No:** 2419158 | **Branch:** CSE - A

1. Project Overview

A relational database that manages students, books, and book issue/return records for a library. It demonstrates table design, primary and foreign keys, constraints, joins, aggregate functions and basic data manipulation (INSERT, UPDATE, DELETE).

2. Files in this Repository

File	Description
`LibraryDB.sql`	Database creation, tables, sample data, queries, DML
`query_output.txt`	Text output of every query
`screenshots/`	Screenshots of query execution and output
`README.md`	This report

Run it with: `mysql -u root -p < LibraryDB.sql` (MySQL / MariaDB).

3. Database Design

Students(student_id PK, roll_no UNIQUE, name, department, email UNIQUE, phone, campus)

Books(book_id PK, title, author, category, price, total_copies, available_copies)

Book_Issue(issue_id PK, student_id FK, book_id FK, issue_date, due_date, return_date, fine)

Relationships: one student can borrow many books and one book can be borrowed by many students, so `Book_Issue` resolves this many-to-many relationship. `return_date` is NULL until the book is returned, so this one table stores both issue and return records.

4. Database Concepts Used

- **Primary key:** uniquely identifies each row (`student_id`, `book_id`, `issue_id`).
- **Foreign key:** `Book_Issue.student_id` and `book_id` reference Students and Books, keeping data consistent (referential integrity). `ON DELETE CASCADE` removes issue records when a student or book is deleted.
- **Constraints:** `NOT NULL`, `UNIQUE` (roll_no, email), `CHECK` (price > 0, available_copies <= total_copies, due_date >= issue_date), `DEFAULT` (campus, fine).
- **Joins:** `JOIN` of three tables to show which student has which book; `LEFT JOIN` to include students with no books.
- **Aggregate functions:** `COUNT`, `SUM`, `AVG`, `MAX` with `GROUP BY` and `HAVING`.
- **DML:** INSERT (issue a book), UPDATE (return a book, change phone), DELETE (remove a student).

5. Key Queries

- Available books: `SELECT ... FROM Books WHERE available_copies > 0;`
- Student details: `SELECT * FROM Students;`

- Books issued to a student: join Book_Issue, Students and Books `WHERE s.roll_no = '2419158'`.
- Pending returns: `WHERE return_date IS NULL`.
- Books issued per student: `LEFT JOIN ... GROUP BY`.

6. Connecting the Database to a Website

A website never talks to the database directly. A three-tier architecture is used:

1. **Frontend** (HTML/CSS/JavaScript or React): the librarian or student fills forms and searches for books.
2. **Backend** (Node.js/Express, PHP, Python Flask/Django or Java Spring): receives HTTP requests, validates input, connects to MySQL with a driver (e.g. `mysql2`, `PyMySQL`, JDBC) using stored credentials, and runs SQL.
3. **Database** (MySQL): executes the query and returns rows, which the backend converts to JSON/HTML for the page.

Example flow: the student enters a roll number -> the browser sends `GET /api/issued-books?roll=2419158` -> the backend runs the join query using a **prepared statement** (`WHERE roll_no = ?`) -> the result is sent back as JSON and shown as a table.

Good practices: use prepared statements to prevent SQL injection, keep DB credentials in environment variables, hash passwords, and give the web app a database user with limited privileges.

7. Distributed Database for Multiple Campuses

If the college has several campuses (Main, North, South), a distributed database stores data at multiple sites that act as one logical database.

- **Fragmentation:** Students and Book_Issue rows can be split horizontally by `campus`, so each campus keeps its own records locally and queries are fast.
- **Replication:** The Books catalogue is copied to every campus so everyone can search all books and a student can borrow at another campus. Replication also gives fault tolerance: if one site fails, others still serve data.
- **Location transparency:** Users query one logical database and the DBMS decides which site holds the data.
- **Consistency:** Updates to shared data (e.g. available copies) use distributed transactions (two-phase commit) or master-slave replication so counts stay correct.
- **Benefits:** faster local access, higher availability, scalability, and each campus can work during network failures.
- **Challenges:** synchronisation, network cost, more complex security and backup.

8. Conclusion

The project implements a normalized library database with enforced integrity, demonstrates the main SQL features, and describes how it can be linked to a web application and scaled across campuses.